



12-DARS · 14 TADAN

React asoslari

Komponentlar, JSX, props, state, hodisalar, ro'yxatlar va key, formalar, useEffect va ma'lumot yuklash, custom hook'lar, Context va React Router.

O'rta · ~150 daqiqa

5-darsdagi Todo ilovasida biz "holat → render" arxitekturasini qo'lda qurdik: holat o'zgarsa, `render()` funksiyasi DOM'ni qayta chizdi. Ilova kattalashgan sari bu qiyinlashadi: qaysi qismni yangilash kerak? Holat bir necha joyda bo'lsa-chi? **React** — aynan shu muammoni hal qiladigan kutubxona: siz *holat qanday ko'rinishga mos kelishini* tasvirlaysiz, React esa DOM'ni o'zi samarali yangilaydi. Bugun React — frontend vakansiyalarining eng katta qismida talab qilinadigan texnologiya.

React'ning asosiy g'oyalari

- **Deklarativlik** — "qanday o'zgartirish" (`appendChild`, `classList.add`) emas, "qanday ko'rinishi kerak" deb yozasiz: `UI = f(state)`.
- **Komponentlar** — interfeys mustaqil, qayta ishlatiladigan bo'laklardan quriladi: `<Header>`, `<ProductCard>`, `<Cart>`.
- **Bir yo'nalishli ma'lumot oqimi** — ma'lumot ota komponentdan farzandga `props` orqali tushadi; farzand o'zgarishni callback orqali "yuqoriga" xabar qiladi.
- **Virtual DOM va reconciliation** — holat o'zgarganda React yangi "virtual" daraxtni eskisi bilan solishtiradi va haqiqiy DOM'ga faqat farqni yozadi.

Loyiha yaratish (Vite)

Terminal

```
npm create vite@latest kitob-client -- --template react
cd kitob-client
npm install
npm run dev          # http://localhost:5173
```

Matn

```
kitob-client/
├── index.html          # yagona HTML sahifa – <div id="root"></div>
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx        # kirish nuqtasi: React'ni #root ga ulaydi
    ├── App.jsx         # ildiz komponent
    ├── index.css
    └── components/
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

JSX

JSX — JavaScript ichida HTML'ga o'xshash sintaksis. Brauzer uni tushunmaydi — Vite uni oddiy funksiya chaqiruvlariga aylantiradi: `<h1 className="t">Salom</h1>` → `React.createElement('h1', { className: 't' }, 'Salom')`.

React (JSX)

```
function Greeting() {
  const user = { name: 'Malika', avatar: '/m.jpg', unread: 3 };
  const isMorning = new Date().getHours() < 12;

  return (
    <div className="greeting">                                {/* class emas — className
*/}
      <img src={user.avatar} alt={user.name} />              {/* {} ichida istalgan JS
ifoda */}
      <h2>{isMorning ? 'Xayrli tong' : 'Salom'}, {user.name}!</h2>
      {user.unread > 0 && <span className="badge">{user.unread}</span>}
      <label htmlFor="q">Qidiruv</label>                      {/* for emas — htmlFor
*/}
      <input id="q" style={{ width: 200, marginTop: 8 }} />  {/* style —
obyekt, camelCase */}
    </div>
  );
}
```

JSX qoidalari:

- Komponent **bitta** ildiz element qaytaradi. Ortiqcha `div` kerak bo'lmasa — fragment: `<...>`.
- Barcha teglar yopilishi shart: `<img />`, `<br />`, `<input />`.
- Atributlar camelCase: `onClick`, `tabIndex`, `className` (lekin `aria-*` va `data-*` o'zgarmaydi).
- `{}` ichida faqat **ifoda** (expression): `if` / `for` emas — ternary, `&&`, `.map()`.
- React `{}` ichidagi matnni avtomatik ekranlaydi — XSS'dan himoya (`dangerouslySetInnerHTML` bundan mustasno, nomi ham ogohlantiradi).

Komponentlar va props

Komponent — **katta harf bilan boshlanadigan** va JSX qaytaradigan funksiya. **Props** — komponentga tashqaridan beriladigan ma'lumotlar (HTML atributlari kabi). Props **faqat o'qish uchun**: komponent o'z props'ini o'zgartirmaydi.

React (JSX)

```
function Avatar({ src, name, size = 48 }) { // destructuring +
  standart qiymat
  return <img className="avatar" src={src} alt={name} width={size} height=
    {size} />;
}

function UserCard({ user, onFollow, children }) {
  return (
    <article className="card">
      <Avatar src={user.photo} name={user.name} size={64} />
      <h3>{user.name}</h3>
      {children}                                {/* teglar orasidagi
mazmun */}
      <button onClick={() => onFollow(user.id)}>Kuzatish</button>
    </article>
  );
}

// Foydalanish
<UserCard user={{ id: 1, name: 'Bobur', photo: '/b.jpg' }} onFollow={(id) =>
  console.log(id)}>
  <p>Frontend dasturchi, Samarqand</p>
</UserCard>
```



Komponentlarga qanday bo'lish kerak?

Dizayn maketiga qarab, har bir mustaqil "blok"ni to'rtburchak bilan o'rab chiqing. Qoida: bitta komponent — bitta vazifa. Komponent 150–200 qatordan oshsa yoki ichida takrorlanuvchi qism bo'lsa — bo'lish vaqti keldi.

State — komponent xotirasi

Oddiy o'zgaruvchi (`let count = 0`) har render'da qaytadan yaratiladi va uning o'zgarishi ekranni yangilamaydi. **State** — React eslab qoladigan va o'zgarganda komponentni **qayta render qiladigan** qiymat.

React (JSX)

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);    // [joriy qiymat, o'zgartiruvchi
  funksiya]

  return (
    <div>
      <p>Bosildi: {count} marta</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)} disabled={count === 0}>No'lga</button>
    </div>
  );
}
```

State bilan ishlashning muhim qoidalar

React (JSX)

```
// 1. State'ni TO'G'RIDAN-TO'G'RI o'zgartirmang – yangi obyekt/massiv yarating
(immutability)
const [todos, setTodos] = useState([]);
todos.push(newTodo); // ❌ React o'zgarishni sezmaydi
setTodos([...todos, newTodo]); // ✅ qo'shish
setTodos(todos.filter(t => t.id !== id)); // ✅ o'chirish
setTodos(todos.map(t => t.id === id ? { ...t, done: !t.done } : t)); // ✅ yangilash

const [form, setForm] = useState({ name: '', email: '' });
setForm({ ...form, email: 'yangi@mail.uz' }); // ✅ obyektning bitta maydonini

// 2. Yangi qiymat eskisiga bog'liq bo'lsa – funksional ko'rinish
setCount(c => c + 1);
setCount(c => c + 1); // ikkalasi ham ishlaydi → +2
// setCount(count + 1); setCount(count + 1); → faqat +1 (ikkalasi ham eski "count" ni ko'radi)

// 3. State yangilanishi asinxron: set'dan keyin darhol eski qiymat
setCount(5);
console.log(count); // hali eski qiymat – yangisi keyingi render'da

// 4. Hisoblanadigan qiymatni state'da saqlamang – render paytida hisoblang
const doneCount = todos.filter(t => t.done).length; // ✅ alohida state emas
```

Hodisalar

React (JSX)

```
function SearchBox({ onSearch }) {
  const [q, setQ] = useState('');

  function handleSubmit(e) {
    e.preventDefault();
    onSearch(q.trim());
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={q}
        onChange={(e) => setQ(e.target.value)}
        onKeyDown={(e) => e.key === 'Escape' && setQ('')}
      />
      <button type="submit">Izlash</button>
    </form>
  );
}

// ⚠️ onClick={handleClick}   ✅ funksiyani BERAMIZ
//   onClick={handleClick()}   ❌ render paytida CHAQIRILADI
```

Shartli render va ro'yxatlar

React (JSX)

```
function ProductList({ products, loading, error }) {
  if (loading) return <Spinner />; // erta qaytish
  if (error) return <p className="error">{error}</p>;
  if (products.length === 0) return <p>Mahsulot topilmadi</p>;

  return (
    <ul className="grid">
      {products.map((p) => (
        <li key={p.id}> {/* key – majburiy va
barqaror */}
          <ProductCard product={p} />
        </li>
      ))}
    </ul>
  );
}
```



key nima uchun kerak?

`key` React'ga ro'yxatdagi qaysi element qaysi ekanini aytadi: element qo'shilsa, o'chirilsa yoki tartib o'zgarsa, React to'g'ri DOM tugunlarini qayta ishlatadi. `key` sifatida **massiv indeksini** ishlatmang (agar ro'yxat tartibi o'zgarsa) — input'lardagi qiymatlar "aralashib ketadi". Ma'lumotning barqaror `id` sini ishlateng.

Formalar: boshqariladigan komponentlar

React (JSX)

```
function RegisterForm({ onSubmit }) {
  const [values, setValues] = useState({ name: '', email: '', plan: 'free',
agree: false });
  const [errors, setErrors] = useState({});

  // Barcha maydonlar uchun bitta ishlovchi – name atributi orqali
  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setValues(v => ({ ...v, [name]: type === 'checkbox' ? checked : value }));
  }

  function validate(v) {
    const e = {};
    if (v.name.trim().length < 2) e.name = 'Ism juda qisqa';
    if (!/^\\S+@\\S+\\.\\S+$/\\.test(v.email)) e.email = 'Email noto\\'g\\'ri';
    if (!v.agree) e.agree = 'Shartlarga rozilik kerak';
    return e;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const errs = validate(values);
    setErrors(errs);
    if (Object.keys(errs).length === 0) onSubmit(values);
  }

  return (
    <form onSubmit={handleSubmit} noValidate>
      <input name="name" value={values.name} onChange={handleChange}
placeholder="Ism" />
      {errors.name && <small className="error">{errors.name}</small>}

      <input name="email" value={values.email} onChange={handleChange}
placeholder="Email" />
      {errors.email && <small className="error">{errors.email}</small>}

      <select name="plan" value={values.plan} onChange={handleChange}>
        <option value="free">Bepul</option>
        <option value="pro">Pro</option>
      </select>
    </form>
  );
}
```

```

    </select>

    <label>
      <input type="checkbox" name="agree" checked={values.agree} onChange=
{handleChange} />
      Shartlarga roziman
    </label>
    {errors.agree && <small className="error">{errors.agree}</small>}

    <button>Ro'yxatdan o'tish</button>
  </form>
);
}

```

Boshqariladigan (controlled) input'da qiymatning yagona "haqiqat manbai" — React state. Shu tufayli istalgan payt qiymatni o'qish, o'zgartirish va validatsiya qilish oson. Katta formalar uchun **React Hook Form** + **Zod** kutubxonalari mashhur.

State'ni yuqoriga ko'tarish (lifting state up)

Ikki komponent bitta ma'lumotdan foydalanishi kerak bo'lsa (masalan, mahsulotlar ro'yxati va savat), state ularning **eng yaqin umumiy otasiga** ko'chiriladi. Ota ma'lumotni props orqali beradi, farzandlar esa o'zgarishni callback orqali so'raydi.

```

function Shop() {
  const [cart, setCart] = useState([]); // umumiy state – otada

  const addToCart = (product) =>
    setCart(c => {
      const found = c.find(i => i.id === product.id);
      return found
        ? c.map(i => i.id === product.id ? { ...i, qty: i.qty + 1 } : i)
        : [...c, { ...product, qty: 1 }];
    });

  return (
    <div className="shop">
      <ProductList onAdd={addToCart} /> /* o'zgartiradi */
      <CartSummary items={cart} /> /* o'qiydi */
    </div>
  );
}

```

useEffect — tashqi dunyo bilan sinxronlash

Render **sof** bo'lishi kerak: faqat JSX hisoblaydi. Server so'rovi, taymer, `document.title`, hodisa tinglovchisi kabi "yon ta'sirlar" (side effects) `useEffect` ichida bajariladi — u render ekranga chiqqandan **keyin** ishlaydi.

React (JSX)

```
import { useEffect, useState } from 'react';

useEffect(() => {
  // ... effekt
  return () => { /* tozalash (cleanup): keyingi effektdan oldin va komponent
o'chganda */ };
}, [dep1, dep2]); // bog'liqliklar massivi

// [] — faqat birinchi render'dan keyin bir marta
// [userId] — birinchi marta va userId o'zgarganda
// massivsiz — HAR render'dan keyin (kamdan-kam kerak)
```

React (JSX)

```
function Clock() {
  const [now, setNow] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id); // tozalanmasa — "xotira oqishi"
    va ikki baravar taymer
  }, []);

  useEffect(() => {
    document.title = now.toLocaleTimeString('uz-UZ');
  }, [now]);

  return <time>{now.toLocaleTimeString('uz-UZ')}</time>;
}
```



StrictMode'da effekt ikki marta ishlaydi

Ishlab chiqish rejimida React tozalash funksiyasi to'g'ri yozilganini tekshirish uchun effektни ataylab "o'rnatish → tozalash → qayta o'rnatish" qiladi. Network panelida ikkita so'rovni ko'rsangiz — bu xato emas. Production'da bir marta ishlaydi.



Effekt kerak bo'lmagan holatlar

Props yoki state'dan hisoblanadigan qiymat uchun effekt + state ishlatmang — shunchaki render paytida hisoblang. Foydalanuvchi harakatiga javob (tugma bosilganda so'rov yuborish) — hodisa ishlovchisida, effektda emas.

Boshqa muhim hook'lar

React (JSX)

```
import { useRef, useMemo, useCallback, useReducer } from 'react';

// useRef — render'lar orasida saqlanadigan, lekin o'zgarganda render
// QILMAYDIGAN qiymat; DOM elementga havola
function AutoFocusInput() {
  const inputRef = useRef(null);
  useEffect(() => inputRef.current.focus(), []);
  return <input ref={inputRef} />;
}

// useMemo — og'ir hisob natijasini keshlash
const sorted = useMemo(
  () => [...products].sort((a, b) => a.price - b.price),
  [products]
);

// useCallback — funktsiyani keshlash (memo'langan farzandga beriladigan
// callback uchun)
const handleAdd = useCallback((p) => setCart(c => [...c, p]), []);

// useReducer — murakkab state mantiqi uchun (Redux uslubi)
function reducer(state, action) {
  switch (action.type) {
    case 'add': return [...state, action.item];
    case 'remove': return state.filter(i => i.id !== action.id);
    case 'clear': return [];
    default: throw new Error(`Noma'lum amal: ${action.type}`);
  }
}

const [items, dispatch] = useReducer(reducer, []);
dispatch({ type: 'add', item: { id: 1, name: 'Kitob' } });
```

Hook qoidalari: (1) faqat komponent yoki boshqa hook'ning **yuqori darajasida** chaqiriladi — `if`, sikl yoki ichki funktsiyada emas; (2) faqat React funktsiyalarida. Sabab: React hook'larni chaqirilish **tartibi** bo'yicha taniydi.

Context — "props drilling"siz ma'lumot uzatish

Joriy foydalanuvchi, til yoki mavzu (theme) kabi ma'lumotlar o'nlab komponentga kerak. Ularni har bir darajadan props orqali o'tkazish ("prop drilling") noqulay. **Context** ma'lumotni daraxtning istalgan chuqurligiga to'g'ridan-to'g'ri yetkazadi.

src/context/ThemeContext.jsx

```
import { createContext, useContext, useEffect, useState } from 'react';

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState(() => localStorage.getItem('theme') ?? 'light');

  useEffect(() => {
    document.documentElement.dataset.theme = theme;
    localStorage.setItem('theme', theme);
  }, [theme]);

  const toggle = () => setTheme(t => (t === 'light' ? 'dark' : 'light'));

  return <ThemeContext.Provider value={{ theme, toggle }}>{children}</ThemeContext.Provider>;
}

export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme ThemeProvider ichida ishlatilishi kerak');
  return ctx;
}

// main.jsx: <ThemeProvider><App /></ThemeProvider>
// Istalgan joyda:
// const { theme, toggle } = useTheme();
// <button onClick={toggle}>{theme === 'light' ? '🌙' : '☀️'}</button>
```

React Router — sahifalar

React ilovasi — **SPA (Single Page Application)**: bitta HTML, sahifalar orasidagi o'tish esa JavaScript bilan, serverga yangi HTML so'ramasdan bajariladi.

Terminal

```
npm install react-router
```

```

import { BrowserRouter, Routes, Route, Link, NavLink, useParams, useNavigate,
Navigate } from 'react-router';

export default function App() {
  return (
    <BrowserRouter>
      <nav>
        <NavLink to="/" end>Bosh sahifa</NavLink>
        <NavLink to="/books">Kitoblar</NavLink>
        <NavLink to="/cart">Savat</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/books" element={<BookList />} />
        <Route path="/books/:id" element={<BookDetails />} />
        <Route path="/cart" element={<RequireAuth><Cart /></RequireAuth>} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

function BookDetails() {
  const { id } = useParams(); // /books/42 → id = '42'
  const navigate = useNavigate();
  // ... id bo'yicha kitobni yuklash
  return <button onClick={() ⇒ navigate(-1)}>< Orqaga</button>;
}

function RequireAuth({ children }) {
  const user = useAuth(); // 13-darsda yoziladi
  return user ? children : <Navigate to="/login" replace />;
}

```

Amaliy misollar

MISOL 1 Mahsulotlar katalogi va savat (komponentlar, props, lifting state)

src/Shop.jsx

```
import { useState } from 'react';

const PRODUCTS = [
  { id: 1, name: 'Simsiz quloqchin', price: 450000, emoji: '🎧' },
  { id: 2, name: 'Aqlli soat', price: 1200000, emoji: '🕒' },
  { id: 3, name: 'Powerbank 20000', price: 320000, emoji: '🔋' },
  { id: 4, name: 'Mexanik klaviatura', price: 890000, emoji: '⌨' },
];

const som = (n) => n.toLocaleString('uz-UZ') + " so'm";

function ProductCard({ product, qtyInCart, onAdd }) {
  return (
    <article className="product">
      <div className="emoji">{product.emoji}</div>
      <h3>{product.name}</h3>
      <p>{som(product.price)}</p>
      <button onClick={() => onAdd(product)}>
        {qtyInCart ? `Savatda: ${qtyInCart} ✓ (+1)` : 'Savatga'}
      </button>
    </article>
  );
}

function CartItem({ item, onChangeQty, onRemove }) {
  return (
    <li className="cart-item">
      <span>{item.emoji} {item.name}</span>
      <div className="qty">
        <button onClick={() => onChangeQty(item.id, item.qty - 1)} aria-label="Kamaytirish">-</button>
        <span>{item.qty}</span>
        <button onClick={() => onChangeQty(item.id, item.qty + 1)} aria-label="Ko'paytirish">+</button>
      </div>
    </li>
  );
}
```

```

        <b>{som(item.price * item.qty)}</b>
        <button onClick={() => onRemove(item.id)} aria-
label="O'chirish">x</button>
      </li>
    );
  }

function Cart({ items, onChangeQty, onRemove, onClear }) {
  const total = items.reduce((s, i) => s + i.price * i.qty, 0);
  // hisoblanadigan qiymat
  const count = items.reduce((s, i) => s + i.qty, 0);
  const delivery = total === 0 || total >= 1_000_000 ? 0 : 25_000;

  if (items.length === 0) return <aside className="cart"><h2>🛒 Savat
bo'sh</h2></aside>;

  return (
    <aside className="cart">
      <h2>🛒 Savat {count}</h2>
      <ul>
        {items.map(item => (
          <CartItem key={item.id} item={item} onChangeQty={onChangeQty}
onRemove={onRemove} />
        ))}
      </ul>
      <p>Yetkazish: {delivery ? som(delivery) : 'BEPUL'}</p>
      {delivery > 0 && <small>Yana {som(1_000_000 - total)} xarid
qilsangiz – bepul yetkazish</small>}
      <h3>Jami: {som(total + delivery)}</h3>
      <button onClick={onClear}>Tozalash</button>
    </aside>
  );
}

export default function Shop() {
  const [cart, setCart] = useState([]);

  const add = (product) => setCart(c =>
    c.some(i => i.id === product.id)
      ? c.map(i => (i.id === product.id ? { ...i, qty: i.qty + 1 } : i))
      : [...c, { ...product, qty: 1 }])
  );

  const changeQty = (id, qty) => setCart(c =>
    qty <= 0 ? c.filter(i => i.id !== id) : c.map(i => (i.id === id ? {

```

```

...i, qty } : i))
);
const remove = (id) => setCart(c => c.filter(i => i.id !== id));

return (
  <div className="shop">
    <section className="products">
      {PRODUCTS.map(p => (
        <ProductCard
          key={p.id}
          product={p}
          qtyInCart={cart.find(i => i.id === p.id)?.qty}
          onAdd={add}
        />
      ))}
    </section>
    <Cart items={cart} onChangeQty={changeQty} onRemove={remove}
onClear={() => setCart([])} />
  </div>
);
}

```

src/index.css (qo'shimcha)

```

.shop { display: grid; grid-template-columns: 1fr 340px; gap: 24px;
padding: 24px; align-items: start; }
.products { display: grid; grid-template-columns: repeat(auto-fill,
minmax(180px, 1fr)); gap: 16px; }
.product { border: 1px solid #e5e7eb; border-radius: 14px; padding: 16px;
text-align: center; }
.emoji { font-size: 48px; }
.cart { position: sticky; top: 16px; border: 1px solid #e5e7eb; border-
radius: 14px; padding: 16px; }
.cart ul { list-style: none; padding: 0; }
.cart-item { display: grid; grid-template-columns: 1fr auto auto auto;
gap: 8px; align-items: center; padding: 6px 0; }
.qty { display: flex; gap: 6px; align-items: center; }
@media (max-width: 800px) { .shop { grid-template-columns: 1fr; } }

```

Bu 4-darsdagi savat mantiqining React versiyasi. Savat state'i `Shop` da, chunki uni ham `ProductCard` (savatda nechta borligini ko'rsatish uchun), ham `Cart` ishlatadi. `total` state emas — har render'da hisoblanadi.

MISOL 2

Todo ilovasi React'da: `useReducer` + `localStorage`

5-darsdagi 150 qatorlik ilovani React'da qayta yozamiz va farqni his qilamiz.

```

import { useEffect, useReducer, useState } from 'react';

function todosReducer(todos, action) {
  switch (action.type) {
    case 'added':
      return [{ id: crypto.randomUUID(), text: action.text, done: false },
        ...todos];
    case 'toggled':
      return todos.map(t => (t.id === action.id ? { ...t, done: !t.done }
        : t));
    case 'edited':
      return todos.map(t => (t.id === action.id ? { ...t, text:
        action.text } : t));
    case 'deleted':
      return todos.filter(t => t.id !== action.id);
    case 'clearedDone':
      return todos.filter(t => !t.done);
    default:
      throw new Error('Noma\'lum amal: ' + action.type);
  }
}

const load = () => {
  try { return JSON.parse(localStorage.getItem('todos-react')) ?? []; }
  catch { return []; }
};

const FILTERS = {
  all: { label: 'Barchasi', fn: () => true },
  active: { label: 'Faol', fn: t => !t.done },
  done: { label: 'Bajarilgan', fn: t => t.done },
};

export default function TodoApp() {
  const [todos, dispatch] = useReducer(todosReducer, undefined, load);
  // lazy init
  const [filter, setFilter] = useState('all');
  const [text, setText] = useState('');

  useEffect(() => {
    localStorage.setItem('todos-react', JSON.stringify(todos));
  }, [todos]);

```

```

const visible = todos.filter(FILTERS[filter].fn);
const left = todos.filter(t => !t.done).length;

function handleSubmit(e) {
  e.preventDefault();
  if (!text.trim()) return;
  dispatch({ type: 'added', text: text.trim() });
  setText('');
}

return (
  <main className="app">
    <h1>📝 Vazifalar</h1>
    <form onSubmit={handleSubmit}>
      <input value={text} onChange={e => setText(e.target.value)}
placeholder="Yangi vazifa..." autoFocus />
      <button disabled={!text.trim()}>Qo'shish</button>
    </form>

    <div className="filters">
      {Object.entries(FILTERS).map(([key, f]) => (
        <button key={key} className={filter === key ? 'active' : ''}
onClick={() => setFilter(key)}>
          {f.label}
        </button>
      ))}
    </div>

    <ul>
      {visible.map(todo => (
        <TodoItem key={todo.id} todo={todo} dispatch={dispatch} />
      ))}
      {visible.length === 0 && <li className="empty">Hozircha vazifa
yo'q 🤖</li>}
    </ul>

    <footer>
      <span>{left} ta faol vazifa</span>
      {todos.some(t => t.done) && (
        <button onClick={() => dispatch({ type: 'clearedDone'
})}>Bajarilganlarni tozalash</button>
      )}
    </footer>
  </main>
)

```

```

    </main>
  );
}

function TodoItem({ todo, dispatch }) {
  const [editing, setEditing] = useState(false);
  const [draft, setDraft] = useState(todo.text);

  function finish(save) {
    if (save && draft.trim()) dispatch({ type: 'edited', id: todo.id,
text: draft.trim() });
    else setDraft(todo.text);
    setEditing(false);
  }

  return (
    <li className={todo.done ? 'done' : ''}>
      <input type="checkbox" checked={todo.done} onChange={() =>
dispatch({ type: 'toggled', id: todo.id })} />
      {editing ? (
        <input
          className="edit"
          value={draft}
          autoFocus
          onChange={e => setDraft(e.target.value)}
          onBlur={() => finish(true)}
          onKeyDown={e => { if (e.key === 'Enter') finish(true); if (e.key
=== 'Escape') finish(false); }}
        />
      ) : (
        <span className="text" onClick={() => setEditing(true)}>
{todo.text}</span>
      )}
      <button onClick={() => dispatch({ type: 'deleted', id: todo.id })}
aria-label="O'chirish">x</button>
    </li>
  );
}

```

Taqqoslang: qo'lda `createElement`, `replaceChildren`, delegatsiya, "qaysi elementni yangilash kerak" degan savollar yo'qoldi. Reducer — sof funksiya: uni React'siz ham test qilish mumkin. Tahrirlash holati (`editing`) har bir element ichida — mahalliy (local) state.

MISOL 3**Ma'lumot yuklash: useFetch custom hook va qidiruv**

Custom hook — `use` bilan boshlanadigan va ichida boshqa hook'lardan foydalanadigan funksiya. U komponentlar orasida **mantiqni** (UI'ni emas) qayta ishlatishga imkon beradi.

src/hooks/useFetch.js

```
import { useEffect, useState } from 'react';

export function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: Boolean(url),
  error: null });

  useEffect(() => {
    if (!url) return;
    const controller = new AbortController();
    setState(s => ({ ...s, loading: true, error: null }));

    fetch(url, { signal: controller.signal })
      .then(res => {
        if (!res.ok) throw new Error(`Server xatosi: ${res.status}`);
        return res.json();
      })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(err => {
        if (err.name !== 'AbortError') setState({ data: null, loading:
false, error: err.message });
      });

    // url o'zgarsa yoki komponent yopilsa – eski so'rovni bekor qilamiz
    (race condition'dan himoya)
    return () => controller.abort();
  }, [url]);

  return state;
}
```

src/hooks/useDebounce.js

```
import { useEffect, useState } from 'react';

export function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}
```

```

import { useState } from 'react';
import { useFetch } from '../hooks/useFetch';
import { useDebounce } from '../hooks/useDebounce';

export default function UserSearch() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 400);
  const { data: users, loading, error } =
useFetch('https://jsonplaceholder.typicode.com/users');
  const [selectedId, setSelectedId] = useState(null);
  const { data: posts, loading: postsLoading } = useFetch(
    selectedId ? `https://jsonplaceholder.typicode.com/posts?
userId=${selectedId}` : null
  );

  const q = debouncedQuery.trim().toLowerCase();
  const filtered = users?.filter(u =>
    u.name.toLowerCase().includes(q) || u.email.toLowerCase().includes(q)
  ) ?? [];

  return (
    <div className="user-search">
      <input
        type="search"
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Ism yoki email bo'yicha qidirish..."
      />

      {loading && <p>Yuklanmoqda... </p>}
      {error && <p className="error">⚠️ {error}</p>}

      <div className="columns">
        <ul>
          {filtered.map(u => (
            <li key={u.id}>
              <button
                className={u.id === selectedId ? 'active' : ''}
                onClick={() => setSelectedId(u.id)}
              >
                <b>{u.name}</b><br /><small>{u.email}</small>
              </button>
            )
          )}
        </ul>
      </div>
    </div>
  );
}

```

```

        </li>
      )}}
      {!loading && filtered.length === 0 && <li>Hech kim
topilmadi</li>}
    </ul>

    <section>
      {!selectedId && <p>Postlarini ko'rish uchun foydalanuvchini
tanlang</p>}
      {postsLoading && selectedId && <p>Postlar yuklanmoqda...</p>}
      {posts?.map(p => (
        <article key={p.id}>
          <h4>{p.title}</h4>
          <p>{p.body}</p>
        </article>
      )}}
    </section>
  </div>
</div>
);
}

```



Real loyihalarda server ma'lumotlarini yuklash uchun **TanStack Query** (React Query) kutubxonasi ishlatiladi: keshlash, qayta yuklash, "stale-while-revalidate", mutatsiyalar va xatolarni boshqarishni tayyor holda beradi. `useFetch` esa uning ishlash tamoyilini tushunish uchun ajoyib mashq.

```
src/hooks/useLocalStorage.js
```

```
import { useEffect, useState } from 'react';

export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const saved = localStorage.getItem(key);
      return saved !== null ? JSON.parse(saved) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    try { localStorage.setItem(key, JSON.stringify(value)); } catch { /*
to'lgan yoki taqiqlangan */ }
    }, [key, value]);

    // Boshqa brauzer yorlig'ida o'zgarsa – sinxronlash
    useEffect(() => {
      const onStorage = (e) => {
        if (e.key === key && e.newValue !== null)
          setValue(JSON.parse(e.newValue));
      };
      window.addEventListener('storage', onStorage);
      return () => window.removeEventListener('storage', onStorage);
    }, [key]);

    return [value, setValue];
  }
}
```

src/context/FavoritesContext.jsx

```
import { createContext, useContext } from 'react';
import { useLocalStorage } from '../hooks/useLocalStorage';

const FavoritesContext = createContext(null);

export function FavoritesProvider({ children }) {
  const [ids, setIds] = useLocalStorage('favorites', []);
  const toggle = (id) => setIds(list => (list.includes(id) ? list.filter(x
=> x !== id) : [...list, id]));
  const isFavorite = (id) => ids.includes(id);
  return (
    <FavoritesContext.Provider value={{ ids, toggle, isFavorite }}>
      {children}
    </FavoritesContext.Provider>
  );
}

export const useFavorites = () => useContext(FavoritesContext);
```

```

import { BrowserRouter, Routes, Route, NavLink, Link, useParams } from
'react-router';
import { FavoritesProvider, useFavorites } from
'./context/FavoritesContext';
import { useFetch } from './hooks/useFetch';

const API = 'https://jsonplaceholder.typicode.com';

function Header() {
  const { ids } = useFavorites(); // props'siz - Context
  orqali
  return (
    <header className="header">
      <NavLink to="/" end>📄 Maqolalar</NavLink>
      <NavLink to="/favorites">★ Saralangan ({ids.length})</NavLink>
    </header>
  );
}

function FavoriteButton({ id }) {
  const { isFavorite, toggle } = useFavorites();
  const fav = isFavorite(id);
  return (
    <button onClick={() => toggle(id)} aria-pressed={fav} title={fav ?
'Saralangan olish' : 'Saralash'}>
      {fav ? '★' : '☆'}
    </button>
  );
}

function PostList({ onlyFavorites = false }) {
  const { data: posts, loading } = useFetch(`${API}/posts?_limit=20`);
  const { ids } = useFavorites();
  if (loading) return <p>Yuklanmoqda...</p>;
  if (!posts) return <p>Maqolalarni yuklab bo'lmadi.</p>;
  const list = onlyFavorites ? posts.filter(p => ids.includes(p.id)) :
posts;
  if (!list.length) return <p>Hozircha saralangan maqola yo'q.</p>;
  return (
    <ul className="posts">
      {list.map(p => (
        <li key={p.id}>

```

```

        <FavoriteButton id={p.id} />
        <Link to={` /posts/${p.id}`}>{p.title}</Link>
      </li>
    )}}
  </ul>
);
}

function PostPage() {
  const { id } = useParams();
  const { data: post, loading, error } = useFetch(`${API}/posts/${id}`);
  const { data: comments } = useFetch(`${API}/posts/${id}/comments`);
  if (loading) return <p>Yuklanmoqda... </p>;
  if (error) return <p>Maqola topilmadi. <Link to="/">Bosh sahifaga</Link>
</p>;
  return (
    <article>
      <Link to="/">< Barcha maqolalar</Link>
      <h1>{post.title} <FavoriteButton id={post.id} /></h1>
      <p>{post.body}</p>
      <h3>Izohlar ({comments?.length ?? '...'})</h3>
      {comments?.map(c => <blockquote key={c.id}><b>{c.email}</b>:
{c.body}</blockquote>)}
    </article>
  );
}

export default function App() {
  return (
    <FavoritesProvider>
      <BrowserRouter>
        <Header />
        <main>
          <Routes>
            <Route path="/" element={<PostList />} />
            <Route path="/favorites" element={<PostList onlyFavorites />}
/>

            <Route path="/posts/:id" element={<PostPage />} />
            <Route path="*" element={<h2>404 – sahifa topilmadi</h2>} />
          </Routes>
        </main>
      </BrowserRouter>
    </FavoritesProvider>
  );
}

```

```
);  
}
```

Yulduzcha bosilganda `Header` dagi hisoblagich ham darhol yangilanadi — garchi ular daraxtning turli shoxlarida bo'lsa ham. Saralanganlar `useLocalStorage` tufayli sahifa yangilanganda ham, boshqa yorliqda ham saqlanadi.

Mashqlar



Mustaqil ish

1. react.dev/learn rasmiy qo'llanmasidagi "Tic-Tac-Toe" o'yinini yasang.
2. 1-misoldagi savatni `CartContext` ga ko'chiring va `useLocalStorage` bilan saqlang; `Header`'da savatdagi mahsulotlar sonini ko'rsating.
3. 6-darsdagi ob-havo ilovasini React'da qayta yozing: `useDebounce`, `useFetch`, yuklanish va xato holatlari bilan.
4. `useOnlineStatus()` hook'ini yozing (`navigator.onLine`, `online` / `offline` hodisalari) va internet uzilganda banner ko'rsating.
5. Ko'p bosqichli forma (wizard): 3 qadam, "Orqaga/Keyingi" tugmalari, har qadamda validatsiya, oxirida umumiy ko'rinish.



Xulosa

- React: `UI = f(state)`; komponentlar — JSX qaytaruvchi funksiyalar.
- Props — tashqaridan, faqat o'qish uchun; state — ichki xotira, o'zgarsa qayta render.
- State'ni o'zgartirmang — yangi massiv/obyekt yarating; hisoblanadigan qiymatni state'da saqlamang.
- Ro'yxatlarda barqaror `key`; formalarda boshqariladigan input'lar.
- `useEffect` — tashqi tizimlar bilan sinxronlash, cleanup bilan; custom hook'lar — mantiqni qayta ishlatish.
- Context — global ma'lumot; React Router — SPA sahifalari.

☐ Darsni o'qib bo'ldim